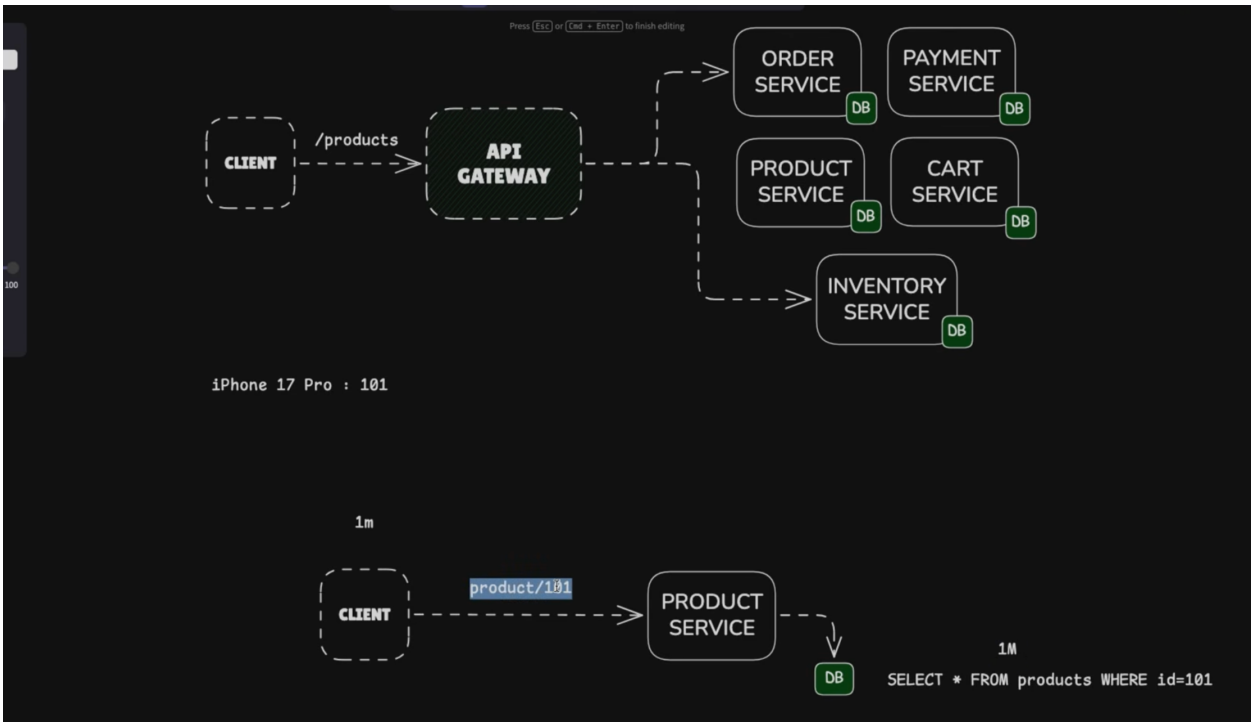
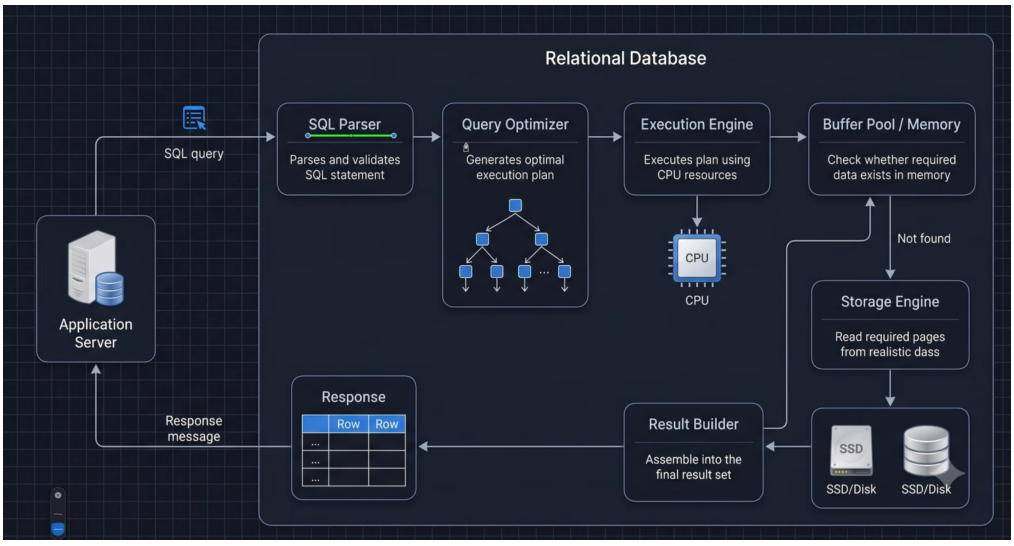
CACHING



1 million users request same product. Product service hit DB 1 million times.

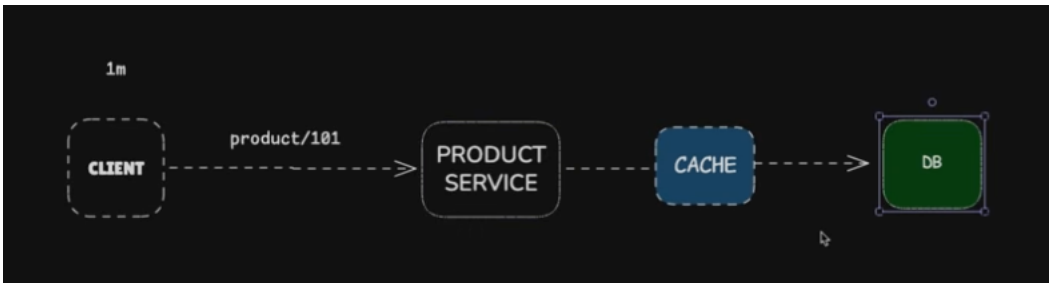
DB is expensive component. DB perform so many operations behind the scenes



Once we execute SQL query – validate the query for errors – generate execution plan – execute query – get response from SSD memory – build the result – response to client.

Assume we have 1 query execute 100ms. What about 1 million queries. Bottle neck.

Solution: CACHE



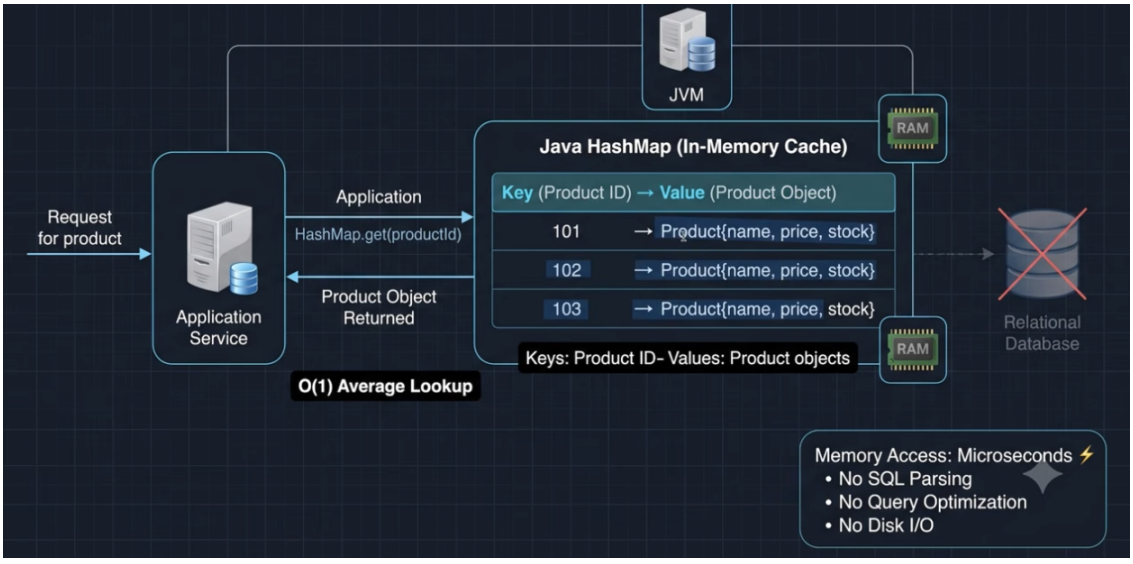

```
Data not found in cache → Cache Miss → Fetch from Database → Store in Cache
Data found in cache → Cache Hit → Return immediately without touching the Database
```

CACHE MISS - hit DB – get data – update the data in cache.

CACHE HIT – if data found in the cache. No hit to DB

Multiple layers of cache we have each solve some concerns.

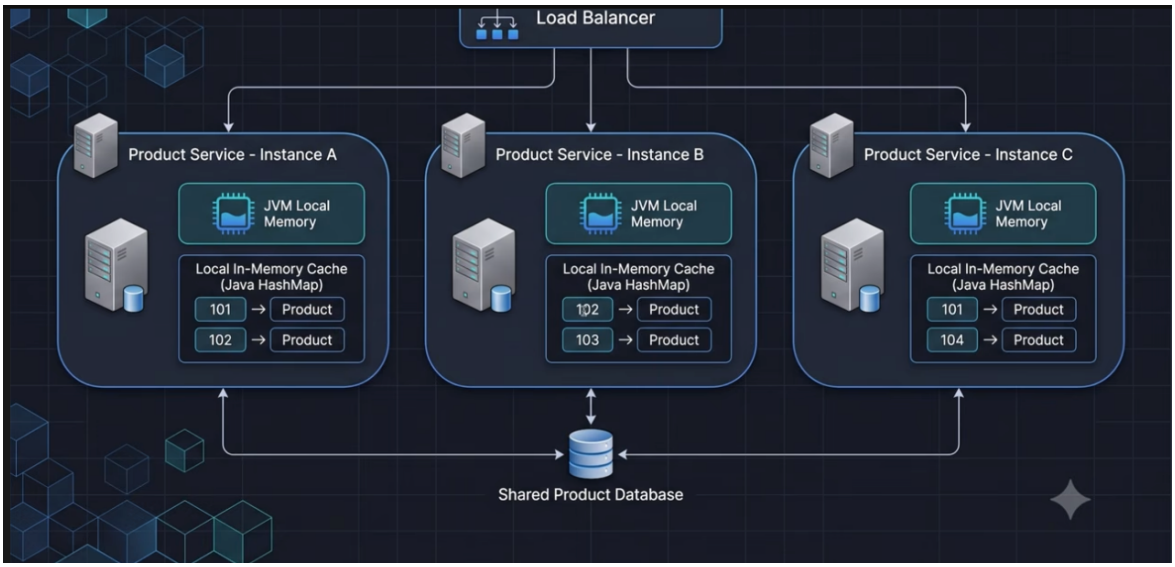
1. **Browser Cache** – Image files – JS – CSS, logo files store browser cache. Not cache dynamic data like price quantity etc.
2. **CDB Cache** – application serves different continent **/** different region - USA (server) – INDIA (client) – images – CSS file – JS – font – videos – load from nearest region
3. **Application Cache** – local cache in HashMap



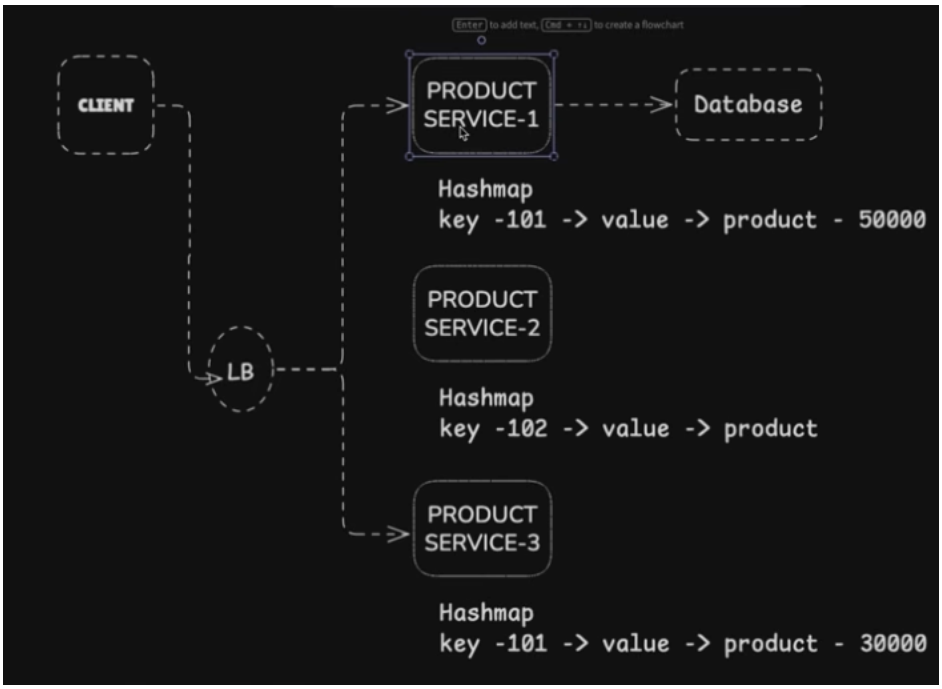
so many problems – what if we have so many instances – each server has its own local copy of HashMap.

What if instance does not have whatever we are looking for. Instance B does not have 101 product info.

Duplicate records across instances.

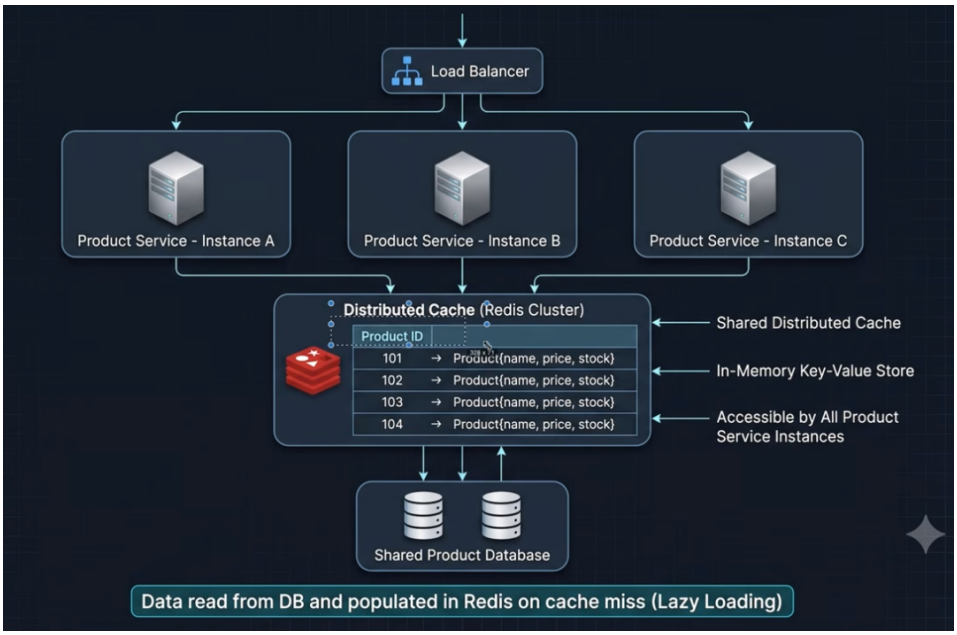


Inconsistency – different prices for same product between instances

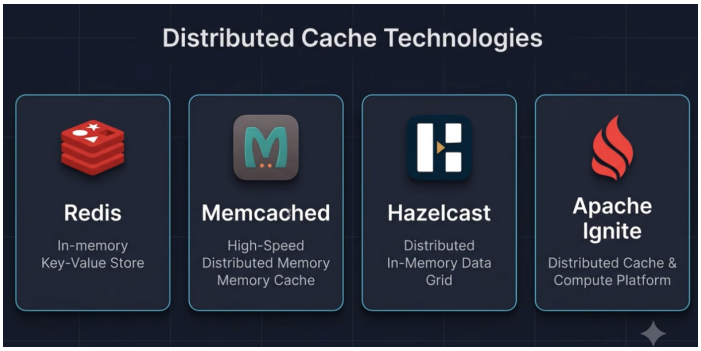


This is useful when we have single instance. To address above challenges we have distributed cache.

4. **Distributed Cache**



Different Distributed Cache.



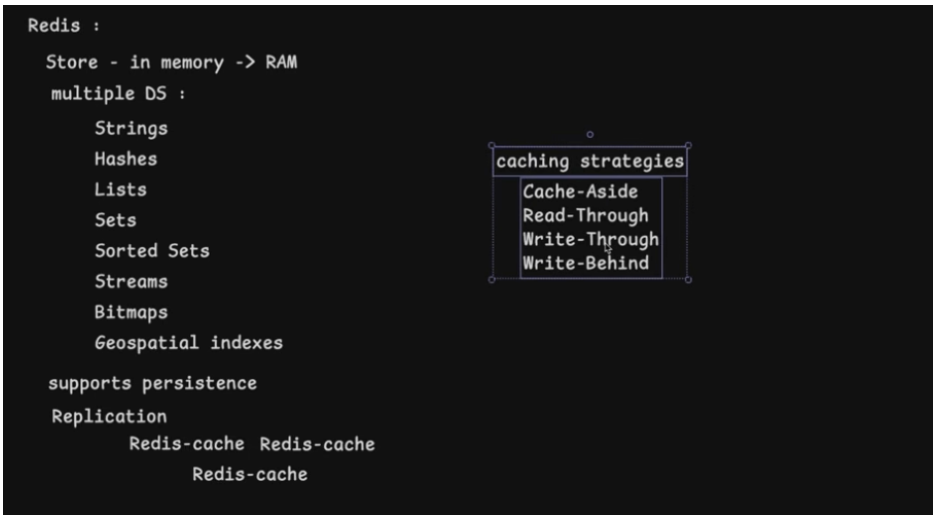
Memcached – store completely in memory – light weight – speed. Simple key value format.

Disadvantages – no persistence – restart data wipeout. No data structure apart from key-value.

HazelCast – only for java application – more configurations require.

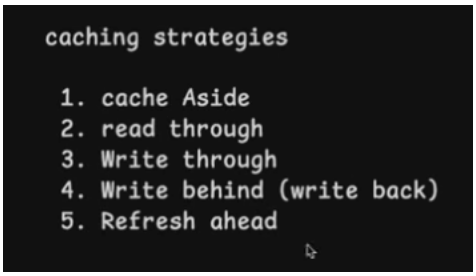
Apache Ignite – in memory computing – distributed cache – SQL cache – but more complex. Specific to use cases.

Redis Cache - Address all the above issues – speed – simplicity – enterprise feature – in memory – RAM only – return in MS – not only key value – multiple Data Structures – support persistence – support replication (multiple Redis cache) – cluster – horizontal scaling – all programming languages – all cloud environment supports.



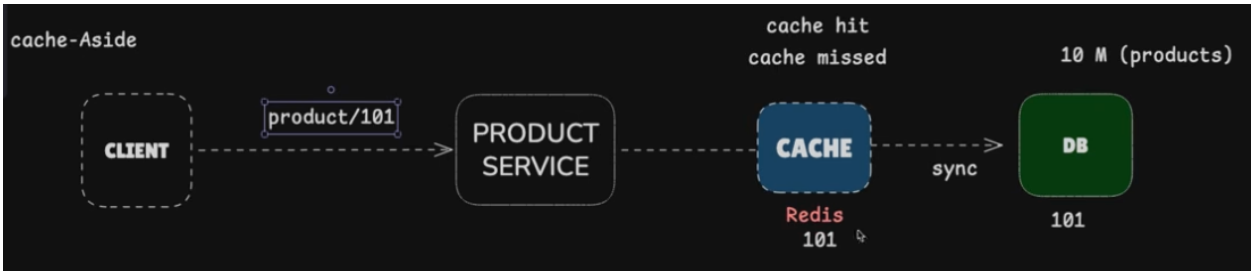
Cache strategy helps

1. When cache and how to Cache and
2. DB should be in sync and consistence and latest.

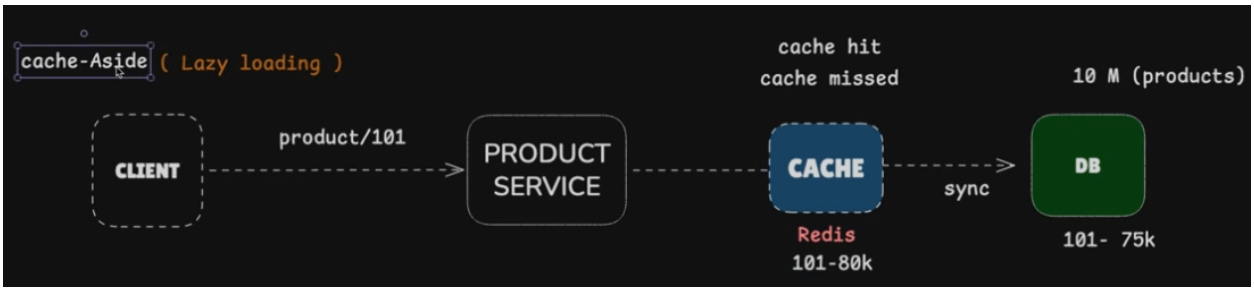


1. **Cache Aside:**
   Application responsibility to update the cache. Deal with How to read from DB and cache. Only frequently accessed data only store in cache. Not all the data.

   When request “cache missed” read from DB and Cached. Then always “cache hit”.



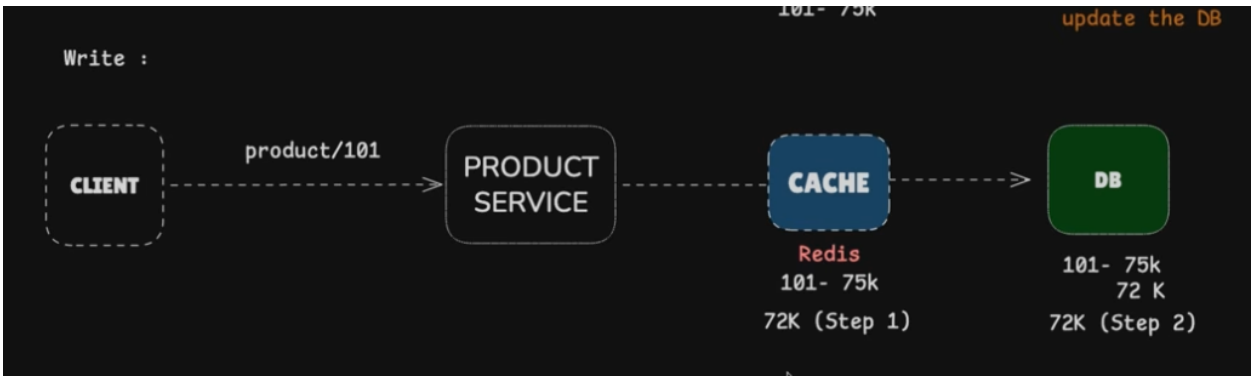
What if DB updated with latest price. DB is showing 75K whereas cache is showing 80K



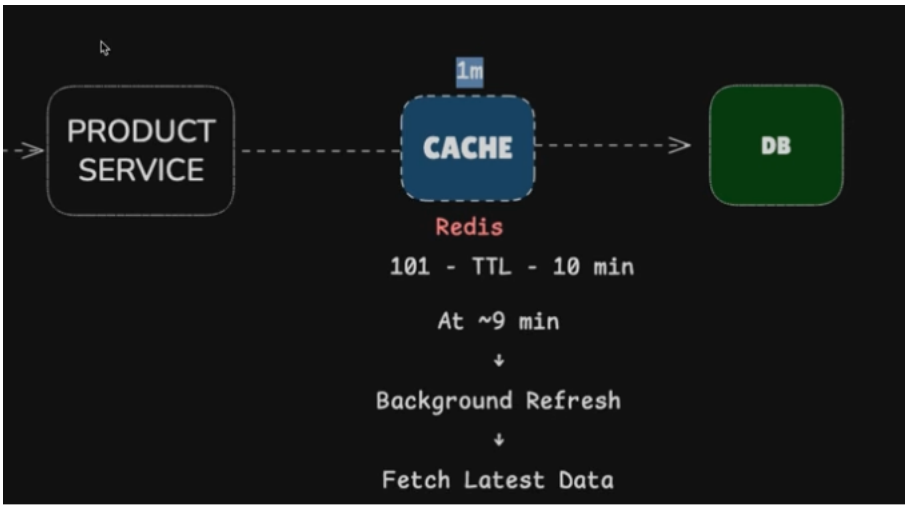
So once update DB then **invalidate** the entry in cache. Delete that entry in the cache. Then it will hit DB and get latest data and cached.

For Ready heavy this is recommended.

2. **Read Though:**
   Cache responsibility to update the cache. Deal with How to read from DB and cache. Invalidate cache will be taken case by cache only.
3. **Write Through:**
   Application responsibility to update cache first and DB second. Deal with how to write to cache and DB. Refer step 1 and step. 2 in the below screenshot. Once complete write in both (cache and DB) then only write will be consider as success. So cache will have updated data.



   Drawback is Higher write latency.
   Unnecessary Cache Data which are never /not frequently used.
4. **Write Behind/ Write Back:** Any update come to application first update Cache first and immediately response to user and asynchronously update DB. Eventually consistence. Problem – what if after writing cache and got response and cache got crashed. Handle failure scenarios very carefully for durability.
5. **Refresh Ahead:** we setup TTL – what if huge traffic after TTL.
   Before it TTL we have to do background refresh only for HOT data like trending products/ popular news/ flash sales Deals . That is what we call it Refresh ahead.



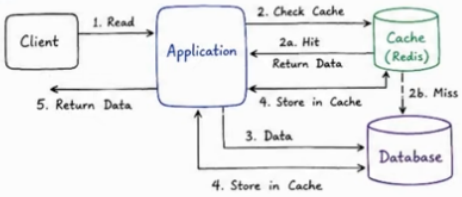
EXPLAIN CACHING STRATEGIES

* Caching strategy decides WHEN to cache, WHAT to cache and HOW to keep the cache fresh and useful.

Goals of Caching
- ✓ Improve response time
- ✓ Reduce load on DB / Services
- ✓ Save bandwidth
- ✓ Improve overall system performance

1) Cache-Aside (Lazy Loading) [Most Common]

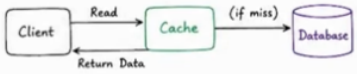
- On read: Check cache first. If data not found, fetch from DB, store in cache and return.
- On write: Update DB first, then invalidate cache.



Use when:
- Read is more frequent than write
- Data can be cached on demand

2) Read-Through

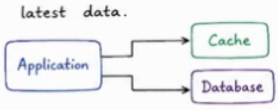
- Application reads through the cache. If data not in cache, cache fetches from DB, stores it and returns.
- Write: Update DB and cache together.



Use when: You want cache to handle retrieval logic.

3) Write-Through

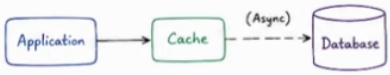
- On write: Write data to cache and DB at the same time.
- Ensures cache always has latest data.



Use when: Data consistency is critical.

4) Write-Behind (Write-Back)

- On write: Update cache only. Cache is written to DB asynchronously (later).
- Improves write performance.



Use when: High write operations and small delay is acceptable.

5) Refresh-Ahead

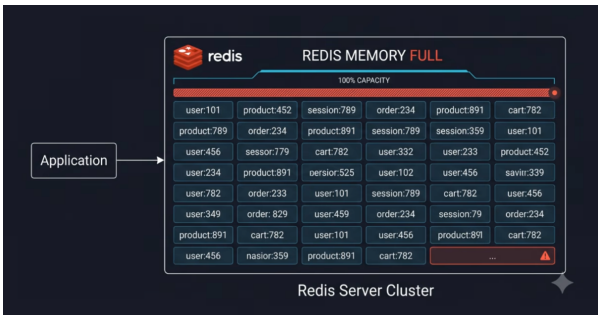
- System proactively refreshes cache before it expires.
- Useful for hot data (frequently accessed).

6) TTL (Time To Live)

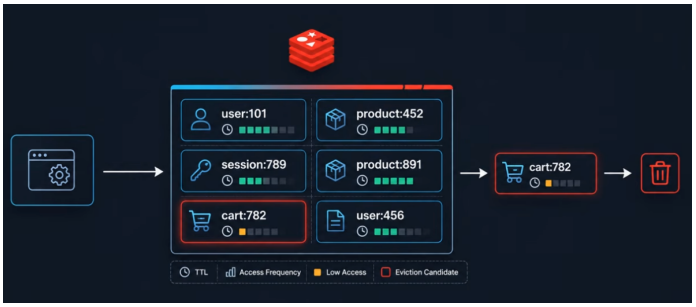
- Set expiry time for cache entries.
- Ensures stale data is removed automatically.

Key Points
- Choose strategy based on data usage pattern.
- Balance between performance and consistency.
- Invalidate or expire stale data properly.

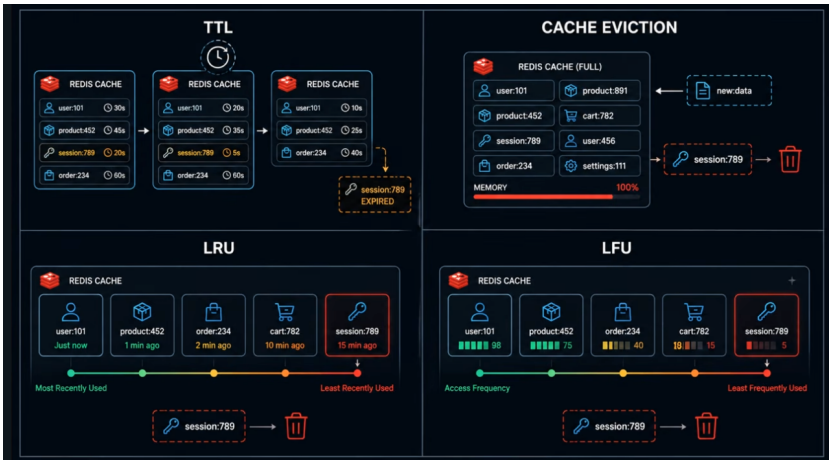
What if Redis cache is full



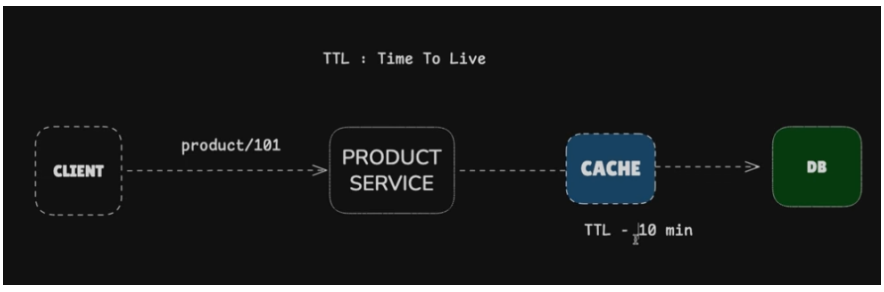
Which one to remove



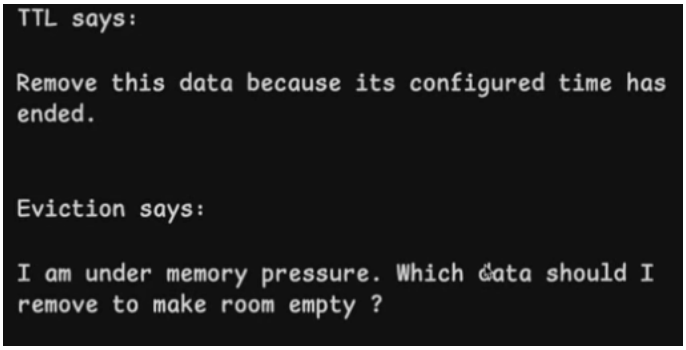
How to manage Chache when it is full.



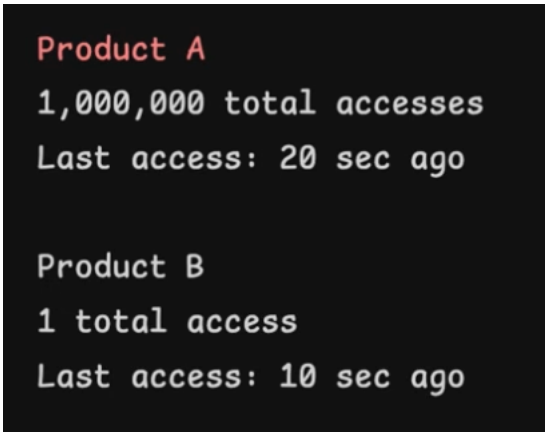
TTL – time to live – how long that object live in the cache After 10 mins Cache missed and hit DB to get Data. No standard to what TTL should be. It depends on scenario.



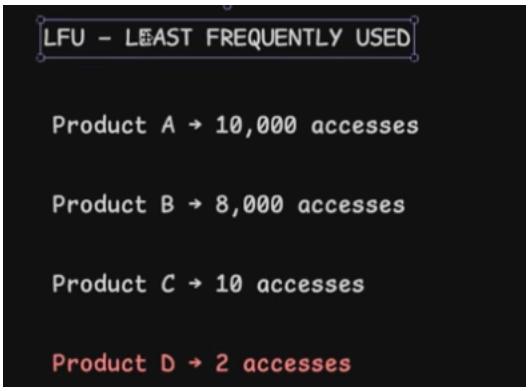
Let us say Max Redis memory capacity 10 GB. What if new data to be cached after full capacity? Solution Cache Eviction come into picture. I am under memory pressure. What should I remove from cache. LRU and LFR.



It uses assumption - Not recently used. So, there may be chance it may not use near feature as well. So, remove from cache.
Problem with assumption – product which as more used got deleted in the below example based LRU.



Solution is LFU



Delete least fequently used. In this case Product D. when our application is based on popularity of product LFU is best.